

Once you have the element, you can query or manipulate it as required. In the preceding code line, you use the `sendKeys()` method to simulate a user typing something into the field. Later in the test, you click on the login button (identified by its `id` attribute) using the `click()` method:

```
driver.findElement(By.id("signin")).click();
```

Finally, at the end of the test, you check the text contents of the welcome message, conveniently identified by an `id` attribute:

```
WebElement welcomeMsg = driver.findElement(By.id("welcome-message"));
assertThat(welcomeMsg.getText()).isEqualTo("Welcome Jane);
```

One of the nice things about this API is that it's not only very readable, but it's very easy to use. In modern IDEs, the auto-complete feature can be used to list the available methods for the various objects and classes used in the WebDriver API (see figure 8.7). This makes the API both easy for new developers to learn and very productive for more experienced developers.

Identifying fields or objects with a given `name` or `id` attribute is the easiest and most robust way to obtain a web element; these attributes are less likely to change when the structure or style of the page changes. In fact, it's a good idea to make sure that all semantically significant elements in a page have a unique ID or name.

This example is relatively straightforward, with the fields and buttons being easy to find. In real-world applications, this isn't always the case, and there are some situations

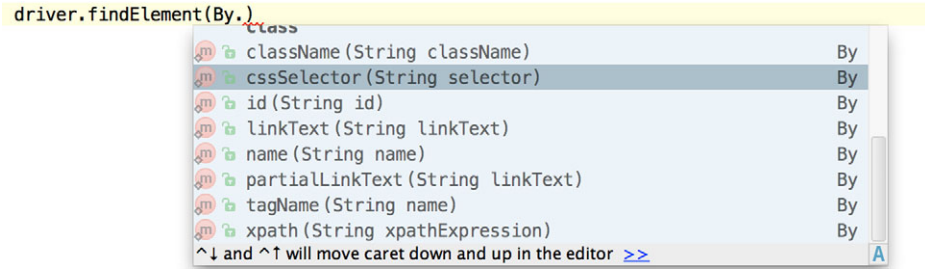


Figure 8.7 Modern IDE features, such as auto-completion, make the WebDriver API easy to work with.